

Dev Saad

Senior Backend Engineer (Go) — distributed systems, order management, options market making

Bengaluru, India | saad.dev07@gmail.com | +91 8989227543 | linkedin.com/in/devsaad07

Summary

Backend engineer, ~5 years (the last four in Go), IIT Jodhpur CSE, building order-management, market-making, and broker-connectivity systems across crypto and Indian equities.

Designed and led, in a two-engineer team, the six-service hedging system behind an options market maker, and built its quoting engine (~10K ops/sec, under 9ms order to execution); designed Lemonn's Go architecture (~250K executed orders/day) and led its eight-engineer vendor migration.

Mentors six engineers, one now owning critical systems independently; made staged canary with a prepared revert the team's default.

Experience

Software Development Engineer II

Apr 2024 – Present

CoinSwitch — one of India's largest crypto investing platforms, Bengaluru, India

Options market making · Mar 2026 – present

- Built the quoting engine behind an in-house crypto options market maker — ~10K order create and cancel operations a second across ~3,000 contracts, order receipt to execution under 9ms end to end: a mirror book merged from source-venue order books with unhedged exposure and lot rounding folded in, quoted by one executor per price level with its own cancel-confirm and orphan-recovery lifecycle.
- Designed and led, with one other engineer, the market maker's production hedging system — six independently deployed Go services that leave no fill unhedged: event-driven over NATS JetStream with PostgreSQL as the source of truth, so any component can crash and rebuild state from the database, and a strategy chain that hedges with options on the source venue first and terminates in futures delta hedging.
- Made the pipeline self-healing across three fill sources (NATS, venue WebSockets, a 30-second REST sweep), deduplicated in memory and on a database unique index so a fill arriving by all three is processed once; a syncer recovers stale and orphaned fills, an independent reconciler audits positions.
- Instrumented the pipeline end to end with Prometheus metrics and Grafana dashboards behind a CI parity gate between dashboard queries and emitted metric names, and put continuous profiling (Pyroscope) across all five market-making engines.
- Held the hedging codebase at 80–95% integration coverage per package behind a CI gate that maps every node of the hedging flow diagram to an asserting test, with race testing on the tracking and reconciliation paths.

Equities broker backend (Lemonn) · Apr 2025 – Mar 2026

- Designed the five-service Go architecture behind Lemonn, CoinSwitch's equities broker (~250K executed orders a day) — order management, funds, portfolio, broker abstraction, shared contracts — replacing the legacy Java middleware, and built the order-management service from its first commit; the patterns the rest of the team now extends.
- Authored the two design documents behind the broker abstraction (multi-vendor routing; the new broker integration): vendor selection, error handling, the Java-to-Go migration plan.
- Led the eight-engineer team that migrated Lemonn to a new broker vendor, from proof-of-concept in mid-2025 to production, as technical point of contact with the vendor's engineering team while aligning risk, brokerage, payments, and frontend stakeholders internally.
- Mentored six engineers through design review and pairing, one of whom now owns critical systems independently, and guided an intern from onboarding to first production deployment.
- Chose Go/CGO for the broker's C++ SDK adapter — now ~400K orders a day with every operation under 5ms at p99 — after benchmarking four wrappers (Go/CGO, native C++, Node.js, Python): Go/CGO stayed timeout-free under load where the native C++ wrapper timed out repeatedly. Kept it a separate service because the SDK's shared library broke Go debugging when linked as a package.

- Drove NATS adoption as the messaging substrate between the broker adapter and order management, phasing it over six months through durable consumers, drain handling, and dead-letter retry rather than a faster cutover that would have forced a second migration.
- Cut user-facing p99 by 87–94% across the order, funds, and portfolio services by tracing a cluster-wide latency pattern to blocking Redis calls in legacy validation jobs starving every consumer of a shared cluster; removing them took Redis GET latency from 3.5–10ms to 0.5ms.
- On-call first responder on Sev1 incidents across order, funds, and payment flows, closing each with a root-cause analysis.
- Made staged canary rollout (1%, 5%, 20%, 100%, revert PR prepared up front) the team's default for every cross-service change; revert path exercised three times in a year.

Crypto exchange platform · Apr 2024 – Mar 2025

- Took ownership of etcd when the organisation had no subject-matter expert, locating misconfiguration by reading upstream source and soak-testing corrections across production clusters; no etcd incident in the months that followed.
- Cut the socket service from 70 pods to 20 by root-causing a Redis memory spike — already escalated as a capacity problem — to per-connection memory held by a holdings Socket.IO subscription and replacing it with a GraphQL query, which also brought the service's CPU under control.

Software Engineer

Sep 2022 – Mar 2024

CoinSwitch, Bengaluru, India

- Part of the reliability programme that cut platform outages ~70%, owning the configuration workstream: took configuration changes — one of three dominant causes mined from past incident reports — out of the deploy path, alongside the programme's ECS-to-EKS migration and end-to-end canary validating every change against 1% of production traffic across 30+ microservices.
- Sole author of the Config Service, a critical-path Go microservice sustaining 450K requests/min and peaking at 1.2M, at a p99.9 of 1.7ms.
- Built the WebSocket event-processing layer on Redis Streams consumer groups, carrying 60K concurrent connections and 400K events/sec at p99 20ms, with weighted load balancing distributing sockets across workers.
- Scaled the shared Redis cluster every service depended on: matched compression to payload instead of one codec everywhere (Snappy on small, frequently read keys, gzip on large ones), split hot keys across servers by prefix, and added an in-process cache to cut round-trips.

Software Engineer, Full Stack

Oct 2021 – Jun 2022

Astute Giraffe, then *ImageKit*, Bengaluru and Remote, India

- ImageKit (Apr 2022 – Jun 2022): shipped multi-workspace support — membership across several accounts with concurrent sessions.
- Astute Giraffe (Oct 2021 – Mar 2022): built the payment system (UPI, net-banking, virtual accounts) behind a modular vendor layer, plus the notification service and admin tooling.

Education

B.Tech, Computer Science and Engineering — *Indian Institute of Technology, Jodhpur*, India Aug 2017 – Jul 2021

Skills

Languages: Go (Golang), TypeScript/Node.js, Java, C++, Python, SQL

Distributed Systems: Microservices, event-driven architecture, message streaming (NATS JetStream, Redis Streams), gRPC and REST APIs, protobuf, CGO, WebSockets, etcd, high availability

Infrastructure & Observability: AWS, Kubernetes (EKS, HPA), Docker, PostgreSQL, Redis, canary deployments, CI/CD, Prometheus, Grafana, pprof and Pyroscope profiling

Practices: System design, HLD authorship, incident response and RCA, integration and race testing, mentoring